



PEMROGRAMAN KOMPUTER 2 (BD322) · WEEK-04

Minggu 04 — Inheritance & Polymorphism

Product memperoleh dua turunan, FoodProduct dan DigitalProduct, yang mengganti satu method tanpa menulis ulang yang lain.

1 Tujuan

Pada akhir minggu ini terdapat tiga jenis produk yang dapat berada dalam satu list dan diperlakukan dengan satu loop yang sama, namun menghasilkan deskripsi yang berbeda-beda. Kemampuan itu belum ada pada Minggu 03, yang hanya mengenal satu jenis produk.

`Product` memperoleh method baru `get_description()` yang mengembalikan nama produk. Method itu menjadi perilaku dasar yang boleh diganti turunannya. `FoodProduct` menggantinya dengan menyertakan tanggal kedaluwarsa, dan `DigitalProduct` dengan menyertakan penanda bahwa produknya tidak berwujud.

`FoodProduct` menambah satu attribute, `expiry_date`. `DigitalProduct` tidak menambah attribute apa pun — yang berbeda hanya perilakunya. Dua bentuk turunan itu ditulis berdampingan untuk memperlihatkan bahwa inheritance tidak selalu berarti menambah data.

Aturan Minggu 03 tidak ditulis ulang di mana pun. `super().__init__()` menjalankan `__init__` milik induknya, sehingga `_price` dan `_stock` beserta seluruh pemeriksaannya berlaku sama pada ketiga jenis produk. `main.py` membuktikannya dengan mengulang percobaan yang harus gagal, kali ini pada sebuah `FoodProduct`.

Batas materi minggu ini. catalog, dictionary sebagai index, struktur data lanjutan.

2 Prasyarat

Minggu 03 sudah ter-merge ke `main` dan tag `week-03` sudah dibuat. Periksa dengan perintah berikut sebelum menulis kode apa pun; bila salah satunya gagal, selesaikan minggu sebelumnya terlebih dahulu.



```
git switch main
git pull
git tag -l week-03
python3 main.py
```

3 Menyiapkan cabang kerja

Minggu ini dikerjakan pada cabang `feature/product-variants`. Cabang dibuat dari `main` yang sudah memuat hasil merge minggu sebelumnya, bukan dari cabang minggu lalu. `git status --short` harus tidak mengeluarkan apa pun: pekerjaan minggu ini dimulai dari working tree yang bersih.

```
git switch main
git pull
git status --short
git switch -c feature/product-variants
```

4 Peta perubahan

Berkas	Perubahan	Keterangan
<code>main.py</code>	UBAH	Satu list berisi tiga jenis produk, satu loop, tiga hasil berbeda.
<code>models/digital_product.py</code>	TAMBAH	Turunan <code>Product</code> yang hanya mengganti perilakunya.
<code>models/food_product.py</code>	TAMBAH	Turunan <code>Product</code> dengan <code>expiry_date</code> dan deskripsi sendiri.
<code>models/product.py</code>	UBAH	Ditambah <code>get_description()</code> sebagai perilaku dasar yang boleh diganti turunannya.

Daftar ini dihasilkan dari repository dengan `git diff --name-status week-03 week-04`.

`README.md` ikut berubah setiap minggu pada bagian Weekly Progress dan dikerjakan pada bagian 10. Berkas kurikulum `PRD.md`, `MILESTONE.md`, `RPS.md`, dan `CLAUDE.md` adalah bahan bacaan, bukan berkas yang ditulis mahasiswa.



5 Langkah kerja

04.1 Menambahkan perilaku dasar pada Product

Method yang akan diganti turunannya harus ada lebih dahulu pada induknya. `get_description()` ditambahkan di akhir class `Product` dan mengembalikan nama produk apa adanya.

Perilaku dasar itu bukan tempat kosong. Sebuah `Product` biasa tetap memberi jawaban yang berguna, sehingga turunan hanya perlu dibuat ketika deskripsinya memang berbeda.

UBAH `models/product.py`

week-04 · baris 33-42

Empat baris terakhir adalah tambahan minggu ini. Bagian di atasnya ditampilkan sebagai penunjuk tempat.

```
33     def reduce_stock(self, quantity):
34         if quantity <= 0:
35             raise ValueError("Quantity must be positive")
36         if quantity > self._stock:
37             raise ValueError("Insufficient stock")
38         self._stock -= quantity
39
40     def get_description(self):
41         # Perilaku dasar. Subclass boleh menggantinya (override).
42         return self.name
```

04.2 Membuat FoodProduct

`class FoodProduct(Product)` menyatakan bahwa `FoodProduct` adalah sebuah `Product`. Seluruh attribute dan method induknya langsung tersedia tanpa disalin.

`__init__` milik turunan menerima satu nilai tambahan, `expiry_date`. Baris `super().__init__(code, name, price, stock)` menyerahkan empat nilai pertama kepada induknya, sehingga `_price` dan `_stock` dibuat oleh kode yang sama dengan Minggu 03 — beserta seluruh aturannya.

Melewatkan `super().__init__()` dan menulis ulang keempat assignment akan menghasilkan program yang tampak berjalan, tetapi menyalin aturan ke tempat kedua. Setiap perubahan aturan kemudian harus dikerjakan dua kali.



TAMBAH models/food_product.py

week-04 · baris 1-17

```
1  """Model FoodProduct – Minggu 04 (Inheritance).
2
3  Produk makanan adalah Product, ditambah satu hal: tanggal kedaluwarsa.
4  """
5
6  from models.product import Product
7
8
9  class FoodProduct(Product):
10     def __init__(self, code, name, price, stock, expiry_date):
11         # super() memanggil __init__ milik Product,
12         # sehingga encapsulation Minggu 03 tetap berlaku.
13         super().__init__(code, name, price, stock)
14         self.expiry_date = expiry_date
15
16     def get_description(self):
17         return f"{self.name} - Expired: {self.expiry_date}"
```

04.3 Membuat DigitalProduct

DigitalProduct tidak menulis `__init__` sama sekali. Karena tidak ada yang perlu ditambahkan, `__init__` milik `Product` dipakai apa adanya, dan object dibuat dengan empat nilai yang sama seperti produk biasa.

Yang ditulis hanya `get_description()`. Sebuah turunan yang isinya satu method adalah turunan yang sah, dan justru memperlihatkan alasan inheritance dipakai: yang berbeda ditulis, yang sama diwarisi.

TAMBAH models/digital_product.py

week-04 · baris 1-12

```
1  """Model DigitalProduct – Minggu 04 (Inheritance).
2
3  Subclass tidak wajib menambah attribute.
4  Di sini yang berbeda hanya perilakunya.
5  """
6
7  from models.product import Product
8
9
10 class DigitalProduct(Product):
11     def get_description(self):
12         return f"{self.name} - Digital Product"
```



04.4 Membuktikan polymorphism dari satu loop

Ketiga jenis produk dimasukkan ke dalam satu list. Loop yang mencetaknya tidak memeriksa jenis object sama sekali — tidak ada `if` yang menanyakan apakah sebuah produk berupa makanan atau produk digital.

Python memilih method yang tepat berdasarkan object yang sedang dipegang. Kemampuan itulah yang disebut polymorphism, dan akibat praktisnya terlihat pada Minggu 15: menambah jenis produk baru tidak mengubah kode yang mencetak daftarnya.

Bagian berikutnya membuktikan pewarisan berjalan dua arah yang diharapkan. `subtotal()` tidak pernah ditulis di `FoodProduct`, tetapi dapat dipanggil; `expiry_date` hanya dimiliki `FoodProduct`, dan tidak ada pada dua jenis lainnya.

UBAH `main.py`

week-04 · baris 11-38

```
11 products = [
12     Product("P001", "Indomie", 3000, 20),
13     FoodProduct("F001", "Roti", 7000, 8, "2026-12-01"),
14     DigitalProduct("D001", "E-Book Python", 50000, 99),
15 ]
16
17 print("=====")
18 print("    SIMPLE CASHIER")
19 print("=====")
20 print()
21
22 print("--- Polymorphism ---")
23
24 # Satu list berisi tiga jenis object, satu loop, tiga hasil berbeda.
25 # main.py tidak perlu tahu jenis produknya.
26 for item in products:
27     print(item.code, "|", item.get_description())
28
29 print()
30 print("--- Yang diwarisi dari Product ---")
31
32 roti = products[1]
33
34 # subtotal() tidak ditulis ulang di FoodProduct, tetapi tetap bisa dipakai.
35 print("Subtotal 3 Roti:", roti.subtotal(3))
36
37 # expiry_date hanya dimiliki FoodProduct.
38 print("Kedaluwarsa Roti:", roti.expiry_date)
```



04.5 Memastikan aturan Minggu 03 tidak rusak

Ketiga percobaan yang harus gagal diulang pada sebuah `FoodProduct` : assignment langsung ke `price` , harga negatif lewat `change_price()` , dan pengurangan stock melebihi persediaan.

Ketiganya tetap ditolak dengan pesan yang sama seperti Minggu 03, meskipun tidak satu baris pun aturan itu ditulis di dalam `models/food_product.py` .

UBAH `main.py`

week-04 · baris 40-62

```
40 print()
41 print("--- Encapsulation Minggu 03 tetap berlaku di subclass ---")
42
43 try:
44     roti.price = 1
45 except AttributeError:
46     print("FoodProduct.price tetap read-only.")
47
48 try:
49     roti.change_price(-1000)
50 except ValueError as error:
51     print("FoodProduct.change_price(-1000) ->", error)
52
53 try:
54     roti.reduce_stock(999)
55 except ValueError as error:
56     print("FoodProduct.reduce_stock(999) ->", error)
57
58 roti.reduce_stock(3)
59 print("Stock Roti setelah terjual 3:", roti.stock)
60
61 print()
62 print("Aturan ditulis sekali di Product, dipakai semua turunannya.")
```

```
git add .
git commit -m "feat: add product variants"
```

Pesan commit di atas adalah pesan yang sungguh dipakai pada repository ini. Daftar lengkapnya diperoleh dengan `git log --no-merges --reverse --format=%s week-03..week-04` .

Commit yang tidak menjadi langkah kerja: `docs: mark week 03 complete in milestone tracker` — pekerjaan dokumentasi yang dikerjakan pada bagian 10.



6 Hasil yang diharapkan

Tiga baris pertama pada bagian *Polymorphism* dihasilkan oleh satu loop yang sama. Perbedaan isinya datang dari method `get_description()` masing-masing jenis produk.

KELUARAN python3 main.py

```
=====
SIMPLE CASHIER
=====

--- Polymorphism ---
P001 | Indomie
F001 | Roti - Expired: 2026-12-01
D001 | E-Book Python - Digital Product

--- Yang diwarisi dari Product ---
Subtotal 3 Roti: 21000
Kedaluwarsa Roti: 2026-12-01

--- Encapsulation Minggu 03 tetap berlaku di subclass ---
FoodProduct.price tetap read-only.
FoodProduct.change_price(-1000) -> Price cannot be negative
FoodProduct.reduce_stock(999) -> Insufficient stock
Stock Roti setelah terjual 3: 5

Aturan ditulis sekali di Product, dipakai semua turunannya.
```

7 Verifikasi

```
python3 main.py
grep -rn "_price\|_stock" models/food_product.py models/digital_product.py
```

- Perintah `grep` tidak boleh mengeluarkan apa pun. Bila ada hasilnya, aturan Minggu 03 disalin ulang ke dalam turunan alih-alih diwarisi.
- Ketiga baris deskripsi pada keluaran berbeda satu sama lain.
- `roti.subtotal(3)` menghasilkan 21000 tanpa `subtotal()` ditulis di `FoodProduct`.
- Ketiga percobaan yang harus gagal tetap ditolak pada object turunan.



Checkpoint MILESTONE.md Minggu 04. inheritance, subclass, overriding, dan polymorphism dapat dijelaskan **dan** dijalankan.

8 Penanganan masalah

Gejala	Sebab	Perbaikan
<code>AttributeError: 'FoodProduct' object has no attribute '_price'</code>	<code>super().__init__()</code> tidak dipanggil, sehingga attribute internal tidak pernah dibuat.	Panggil <code>super().__init__(code, name, price, stock)</code> sebagai baris pertama di dalam <code>__init__</code> turunan.
<code>TypeError: __init__() takes 5 positional arguments but 6 were given</code>	<code>DigitalProduct</code> dibuat dengan lima nilai, padahal ia memakai <code>__init__</code> milik <code>Product</code> yang hanya menerima empat.	Buat <code>DigitalProduct</code> dengan empat nilai, atau tulis <code>__init__</code> sendiri bila memang perlu attribute tambahan.
Ketiga produk mencetak deskripsi yang sama	Nama method pada turunan berbeda dari nama method induknya, misalnya <code>getDescription()</code> atau <code>description()</code> .	Samakan nama dan jumlah parameternya. Override hanya terjadi bila namanya sama persis.
<code>ImportError: cannot import name 'Product'</code>	Berkas turunan mengimport dari dirinya sendiri, atau nama modulnya salah ketik.	Gunakan <code>from models.product import Product</code> di dalam <code>models/food_product.py</code> dan <code>models/digital_product.py</code> .
<code>expiry_date</code> dapat dibaca dari produk biasa	<code>expiry_date</code> ditambahkan pada <code>Product</code> , bukan pada <code>FoodProduct</code> .	Pindahkan attribute itu ke turunannya. Yang dimiliki semua produk berada di induk; yang khusus berada di turunan.



9 Pembandingan dengan checkpoint

Hasil pekerjaan minggu ini dibandingkan dengan checkpoint resmi pada tag `week-04` . Perintah berikut dijalankan dari repository yang sudah memuat tag tersebut; keluaran kosong berarti berkasnya identik.

```
git diff week-04 -- main.py
git diff week-04 -- models/digital_product.py
git diff week-04 -- models/food_product.py
git diff week-04 -- models/product.py
```

Perbedaan penamaan variabel dan urutan fungsi bukan kesalahan. Yang diperiksa adalah perilaku programnya sama dan konsep minggu ini benar-benar dipakai.

Perhatikan panjang kedua berkas turunan: tujuh belas dan dua belas baris. Turunan yang jauh lebih panjang dari itu biasanya menyalin ulang apa yang seharusnya diwarisi.

Pertanyaan yang harus dapat dijawab secara lisan: mengapa inheritance dipakai di sini, apa yang dikerjakan `super()` , dan mengapa loop pencetak daftar tidak perlu tahu jenis produknya.

10 Menutup minggu

Perbarui bagian Weekly Progress pada `README.md` dengan satu baris untuk Minggu 04, lalu dorong cabang dan buka Pull Request berjudul **Week 04 — Add Product Variants**.

```
git add README.md
git commit -m "docs: mark week 04 complete in milestone tracker"
git push -u origin feature/product-variants
```

Isi deskripsi Pull Request memakai template `MILESTONE.md` . Kolaborasi Git bernilai 15 % pada rubrik `PRD.md` §16, dan deskripsi PR adalah bukti yang dinilai.

```
1  ## What
2  ## Why
3  ## How
4  ## Testing
5  ## Screenshot / Evidence
6  ## Learning Point
```



Setelah PR di-review dan di-merge, tarik `main` yang sudah terbaru lalu beri tag `week-04`. Tag inilah yang membuat Minggu 04 dapat dibuka kembali kapan pun.

```
git switch main
git pull
git tag week-04
git push origin week-04
```